

Relazione homework 1

Gruppo di lavoro:

Roberta Vallelunga (Matricola: 1000003065)

Antonio Zarbo (Matricola: 1000081002)

Simone Squillaci (Matricola: 1000081013)

1. Descrizione del progetto:

Il sistema realizzato è un'architettura distribuita a microservizi progettata per gestire utenti e monitorare il traffico aereo basato su specifici interessi. L'applicazione permette agli utenti di registrarsi e selezionare aeroporti di interesse (identificati tramite codice ICAO).

Un componente dedicato si occupa di raccogliere periodicamente i dati sui voli (partenze e arrivi) dagli aeroporti selezionati utilizzando l'API esterna OpenSky Network, archiviando le informazioni per successive consultazioni e analisi statistiche.

Il sistema è logicamente suddiviso in due domini funzionali:

- Gestione Utenti: Registrazione, eliminazione degli utenti e verifica dell'identità.
- Collezione Dati: Gestione degli interessi (aeroporti), interazione con API esterne per il recupero delle informazioni sui voli e calcolo di statistiche sui voli.

2. Scelte progettuali e architettura

L'architettura adotta il pattern a Microservizi, implementata tramite container Docker orchestrati con Docker Compose. Questa scelta garantisce un elevato disaccoppiamento, scalabilità indipendente dei componenti (evidenziata dall'uso di database separati per ogni servizio) e isolamento dei dati.

Componenti Principali (Microservizi)

Il sistema si compone di due microservizi principali sviluppati in Python:

- UserManager:
 - Responsabilità: Gestisce il ciclo di vita degli utenti (registrazione e cancellazione) e garantisce l'idempotenza delle richieste di registrazione (politica At-Most-Once) tramite un identificativo univoco (messageID).
 - Database: Utilizza un database MySQL dedicato (userDB) contenente le tabelle users e requestID. La persistenza del requestID sul DB garantisce maggiore

robustezza.

La scelta di utilizzare MySQL è stata presa considerando la dimestichezza con l'ambiente.

- Interfacce: Espone API REST per le operazioni utente e un server gRPC per rispondere alle verifiche di esistenza richieste internamente dal DataCollector.
- DataCollector:
 - Responsabilità: Gestisce la lista degli aeroporti di interesse per ogni utente, interroga l'API OpenSky e fornisce statistiche. Integra uno scheduler (APScheduler) che esegue l'aggiornamento dei dati in background ogni 12 ore.
 - Database: Utilizza un database MySQL dedicato (dataDB) con le tabelle interests e flights.
 - Interfacce: Espone API REST per i client e agisce come client gRPC verso lo UserManager per validare le richieste (es. verificare che l'utente esista prima di aggiungere un interesse).

Comunicazione e Reti

Il sistema utilizza un modello di comunicazione ibrido. Per le comunicazioni verso l'esterno, ovvero quelle tra i client (es. Postman) e i microservizi, così come per l'interrogazione delle API di terze parti (*OpenSky Network*), è stato utilizzato lo standard REST. Questa scelta garantisce interoperabilità e facilità di utilizzo per gli utenti finali.

Al contrario, per la comunicazione interna inter-service, il sistema implementa il framework gRPC, utilizzato per garantire la comunicazione sincrona e a bassa latenza tra microservizi. Nello specifico, gRPC viene utilizzato dal DataCollector per interrogare l'UserManager e verificare l'esistenza di un utente prima di aggiungere un interesse o fornire dati sui voli.

L'intera architettura è orchestrata tramite Docker Compose che permette una segmentazione logica della rete per massimizzare la sicurezza e l'isolamento dei dati. Invece di utilizzare un'unica rete condivisa, sono state definite tre reti distinte. La privacy dei dati è garantita dalle reti userNet e dataNet, le quali collegano ciascun microservizio esclusivamente al proprio database di riferimento, rendendoli inaccessibili agli altri componenti. La comunicazione tra i due microservizi applicativi avviene invece su una terza rete dedicata, denominata internalNet, che funge da *bridge* permettendo lo scambio di messaggi gRPC senza esporre i database.

Gestione della Concorrenza

La capacità del sistema di gestire richieste multiple simultaneamente, garantendo efficienza e scalabilità, è stata affrontata su due livelli distinti: l'accesso ai dati e l'esecuzione dei processi applicativi.

Per quanto riguarda il primo, entrambi i microservizi implementano un meccanismo di Connection Pooling verso il database MySQL. Invece di aprire e chiudere una nuova

connessione TCP per ogni singola query (operazione che introdurrebbe una latenza significativa) i servizi mantengono attivo un pool di connessioni pronte all'uso (configurato con una dimensione fissa di 10). Questo approccio permette ai servizi di "prendere in prestito" e rilasciare rapidamente le risorse di connessione, massimizzando il throughput delle operazioni di lettura e scrittura.

A livello applicativo, la concorrenza è gestita sfruttando il **multithreading**, una scelta architetturale indispensabile per il funzionamento ibrido del microservizio *UserManager*. Dovendo questo componente esporre contemporaneamente API REST (per le richieste utente) e interfacce gRPC (per i controlli interni), non poteva operare in modo sequenziale. L'esecuzione del server gRPC è stata quindi delegata a un thread separato (*daemon thread*) avviato parallelamente al processo principale Flask. Inoltre, lo stesso server gRPC utilizza internamente un `ThreadPoolExecutor`, assicurando che le richieste di verifica provenienti dal *DataCollector* vengano elaborate in parallelo.

Database

I database sono creati in due container differenti per garantire una scalabilità indipendente tra i due e il completo disaccoppiamento dei dati. La configurazione è definita all'interno del file `docker-compose.yaml`. Qui sono indicate le porte esterne di collegamento, i volumi, per assicurare che i dati non vadano persi in caso di riavvio o crash dei container, e le reti private per la connessione con i rispettivi microservizi.

I due database sono composti da due tabelle ciascuno:

dataDB

- tabella `interests`, dove vengono inseriti gli aeroporti di interesse (tramite ICAO) degli utenti:

	 <code>id</code>	 <code>email_user</code>	 <code>cod_aeroporto</code>
1	2	nero@example.com	KATL
2	6	nero@example.com	LEMD
3	7	nero@example.com	LIMC
4	1	nero@example.com	OMDB

- tabella flights, dove vengono inseriti i voli scaricati da OpenSky relativi agli aeroporti aggiunti nella tabella interests:

	id	icao_24	icao_partenza	icao_arrivo	orario_partenza	orario_arrivo
1	1414	a3a8c0	KIAD	KATL	2025-11-28 11:15:18	2025-11-28 12:38:39
2	1415	a46e6a	KMCO	KATL	2025-11-28 11:05:47	2025-11-28 12:35:18
3	1416	440057	LFRS	LEMD	2025-11-28 09:59:47	2025-11-28 11:11:59
4	1417	34635a	KMIA	LEMD	2025-11-28 03:24:21	2025-11-28 11:09:01
5	1418	a59601	KORD	LIMC	2025-11-28 03:41:11	2025-11-28 11:31:19
6	1419	3c7865	LIRQ	LIMC	2025-11-28 10:32:30	2025-11-28 11:00:55
7	1420	4cadef	EGCC	LIMC	2025-11-28 08:50:17	2025-11-28 10:44:10
8	1421	3c648a	EDDF	LIMC	2025-11-28 06:20:35	2025-11-28 07:16:07
9	1422	4cadb9	<null>	LIMC	2025-11-28 05:53:41	2025-11-28 07:06:51
10	1423	a1775f	KATL	<null>	2025-11-28 14:54:35	2025-11-28 14:56:34
11	1424	ada16f	KATL	<null>	2025-11-28 14:53:24	2025-11-28 14:56:34
12	1425	a2a213	KATL	<null>	2025-11-28 14:53:14	2025-11-28 14:56:34
13	1426	acad53	KATL	<null>	2025-11-28 14:51:02	2025-11-28 14:56:34
14	1427	a3a8c0	KATL	<null>	2025-11-28 14:50:23	2025-11-28 14:56:34
15	1428	a399de	KATL	<null>	2025-11-28 14:50:19	2025-11-28 14:56:34

userDB

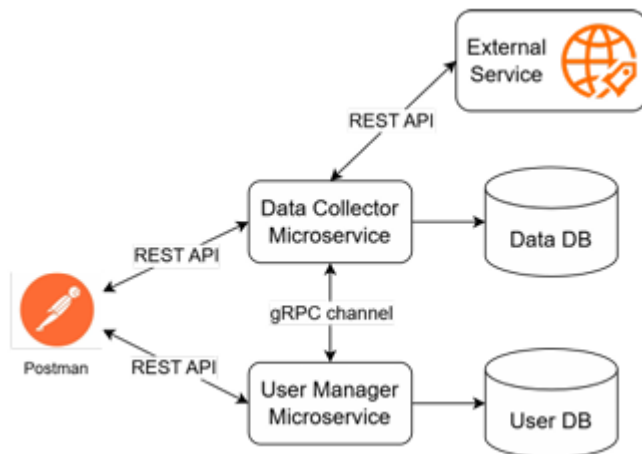
- tabella users, contiene gli utenti registrati al servizio

	email	nome	cognome
1	nero@example.com	Mario	Nero

- tabella request, fondamentale per la politica *At-Most-Once*. Memorizza l'ID della richiesta e l'esito associato, impedendo la ri-esecuzione di richieste di registrazione identiche

	messageID	response
1	1be2352c-193c-4160-8042-e6ccf5867427	Utente già registrato
2	67515f3a-2ced-4d61-8a9e-be2aa1b2ed1a	Richiesta elaborata con successo
3	e6907322-cffd-4a04-9156-8fb701351030	Richiesta elaborata con successo

Schema Architeturale



3. Lista API implementate

UserManager (Porta utilizzata: 5001)

Metodo	Endpoint	Descrizione
POST	/new	Registra un nuovo utente. Richiede messageId nel body per il controllo di idempotenza (<i>At-Most-Once</i>).
DELETE	/delete	Cancella un utente dal database.
gRPC	UserExists	Procedura remota chiamata internamente dal DataCollector per verificare l'esistenza di un utente tramite email.

DataCollector (Porta utilizzata: 5002)

Metodo	Endpoint	Descrizione
POST	/add	Aggiunge un interesse (aeroporto) per un utente. Chiama il gRPC (attraverso <code>check_user_exists_grpc</code>) per verificare se l'utente è registrato al servizio prima di avviare l'operazione di aggiunta dell'interesse nel database.

GET	/view_interests	Permette all'utente di visualizzare i suoi aeroporti di interesse.
DELETE	/delete_interest	Rimuove un aeroporto dagli interessi dell'utente.
GET	/get_data	Trigger manuale che permette di aggiornare forzatamente le informazioni sui voli di interesse degli utenti. Utilizzata in fase di testing per verificare l'effettivo funzionamento dell'API e l'aggiunta nel database.
GET	/view_flights	Restituisce i voli salvati nel DB per un dato aeroporto, utilizzata in fase di testing per verificare l'effettiva aggiunta delle informazioni sui voli.
GET	/last_flights	Restituisce l'ultimo volo (arrivo/partenza) registrato per un aeroporto di interesse dell'utente.
GET	/avg_flights	Calcola la media dei voli giornalieri su un arco temporale di n giorni (parametro query).

Esiste anche una versione modificata di `get_data` (`get_data_scheduler`) invocata automaticamente ogni 12 ore dal `BackgroundScheduler`. Questa variante aggiorna massivamente i dati di volo per tutti gli interessi attivi, garantendo statistiche recenti senza intervento manuale.

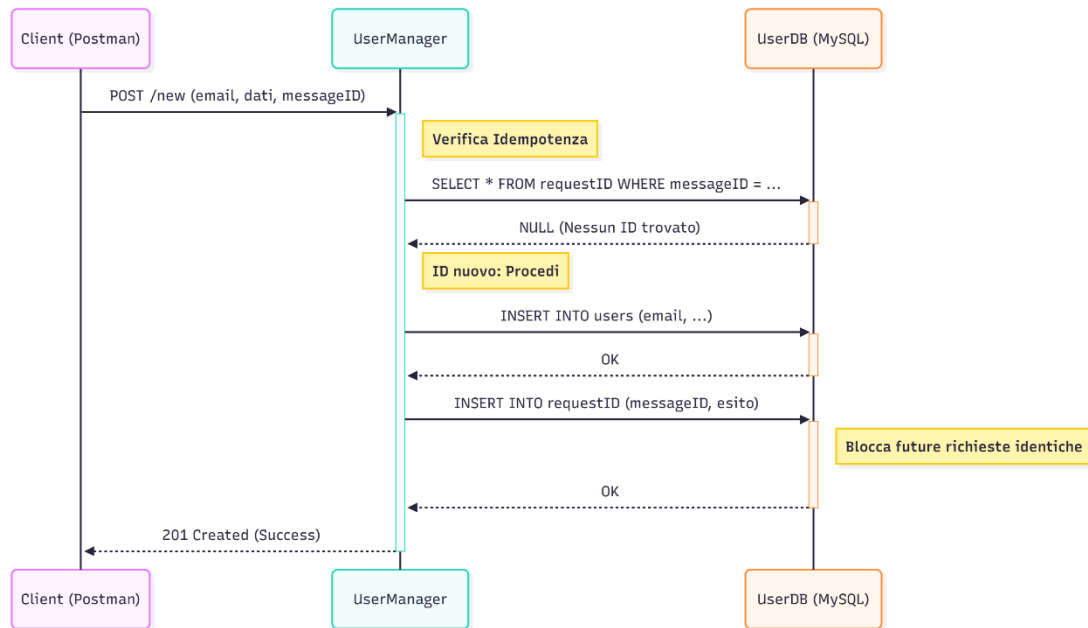
NOTA: Per garantire la consistenza dei dati, tutti gli endpoint operativi replicano la logica di sicurezza di `/add`, eseguendo una verifica preliminare dell'utente tramite chiamata `gRPC (UserExists)` prima di ogni operazione.

4. Dettaglio delle Interazioni (Diagrammi di Sequenza)

Flusso di registrazione con politica At-Most-Once.

Il diagramma illustra la logica di idempotenza implementata nello *User Manager*. All'arrivo di una richiesta POST, il sistema interroga preliminarmente la tabella `requestID` sul database. Solo se il `messageID` univoco fornito dal client non è presente, il sistema procede con l'inserimento dell'utente e il salvataggio del nuovo ID transazione. Questo garantisce che

eventuali ritrasmissioni della stessa richiesta (es. per timeout di rete) non generino duplicati o stati inconsistenti.



Interazione REST e gRPC per l'aggiunta di un interesse.

Il diagramma di sequenza evidenzia la natura ibrida della comunicazione nel sistema. Il client interagisce con il *DataCollector* tramite una chiamata REST standard. Tuttavia, prima di persistere il dato, il *DataCollector* agisce da client gRPC invocando la procedura remota *UserExists* sullo *UserManager*. Questa validazione sincrona *inter-service* assicura l'integrità referenziale dei dati distribuiti su database diversi, impedendo l'inserimento di interessi per utenti non registrati.

